



# Capa de identidad institucional en SIIAA Laravel 13

## Objetivo

La capa de identidad institucional permite que los módulos funcionales del SIIAA trabajen con un identificador común para distintos tipos de usuarios institucionales.

En lugar de asociar registros directamente a:

```
users.id  
personas.id  
estudiantes.id
```

se usa:

```
identity_links.id
```

De esta forma, un registro puede pertenecer de manera uniforme a:

```
Persona SIIAA  
Estudiante SIIAP  
Otro tipo de identidad futura
```

## 1. Concepto central

La tabla principal de esta capa es:

```
identity_links
```

Cada registro representa una identidad institucional operativa.

Ejemplos:

```
identity_links.id = 10000025  
identity_type = siiiaa  
identity_id = personas.id
```

```
identity_links.id = 20000365  
identity_type = siiap_student  
identity_id = estudiantes.id
```

Por tanto:

```
identity_links.id
```

es el identificador común que deben usar los módulos para determinar propiedad institucional.

## 2. Relación con **owner\_id**

Para módulos funcionales futuros, se acordó que el campo:

```
owner_id
```

debe apuntar lógicamente a:

```
identity_links.id
```

No debe apuntar a:

```
users.id
personas.id
estudiantes.id
```

salvo que un módulo documente explícitamente otra relación.

## Regla general

```
owner_id = identity_links.id
```

Esto permite que el propietario institucional de un registro sea una persona SIIAA o un estudiante SIIAP sin cambiar la estructura del módulo.

## 3. Diferencia entre `user_id`, `owner_id`, `created_by` y `updated_by`

| Campo                   | Significado                                      |
|-------------------------|--------------------------------------------------|
| <code>user_id</code>    | Cuenta de acceso al sistema                      |
| <code>owner_id</code>   | Identidad institucional propietaria del registro |
| <code>created_by</code> | Usuario que creó el registro                     |
| <code>updated_by</code> | Usuario que actualizó el registro                |

Ejemplo:

```
owner_id = identidad institucional del estudiante
created_by = usuario administrador que creó el registro
updated_by = usuario editor que modificó el registro
```

Esto permite que un administrador edite información de otra identidad sin alterar la propiedad institucional del registro.

## 4. Proceso general de resolución de identidad

El flujo esperado es:

```
Usuario autenticado
↓
ResolveCurrentIdentity middleware
↓
Búsqueda en identity_links
↓
Identidad activa/resuelta
↓
identity_link_id disponible para la sesión
↓
Uso en módulos mediante owner_id
```

En términos funcionales:

1. El usuario inicia sesión.
2. El middleware ResolveCurrentIdentity toma el correo del usuario.
3. Busca una identidad activa en identity\_links.
4. Resuelve si el usuario corresponde a:
  - persona SIIAA
  - estudiante SIIAP
5. Guarda o deja disponible la identidad institucional activa.
6. Los módulos usan esa identidad para crear o filtrar registros.

## 5. Identidad activa en sesión

Debe existir una forma centralizada de obtener el ID de identidad activa del usuario.

La opción recomendada es normalizarlo como:

```
session('identity_link_id')
```

Sin embargo, para no depender directamente del nombre de la sesión en todos los módulos, se recomienda usar helpers globales.

---

## 6. Helper recomendado:

### **activeIdentityLinkId()**

### Objetivo

Obtener el ID de la identidad institucional activa del usuario autenticado.

```
if (! function_exists('activeIdentityLinkId')) {  
    function activeIdentityLinkId(): ?int  
    {  
        return session('identity_link_id');  
    }  
}
```

Uso:

```
$ownerId = activeIdentityLinkId();
```

## 7. Helper recomendado:

### **currentIdentityLink()**

### Objetivo

Obtener el modelo completo de la identidad activa.

```
if (! function_exists('currentIdentityLink')) {  
    function currentIdentityLink(): ?\App\Models\IdentityLink  
    {  
        $identityId = activeIdentityLinkId();  
  
        if (! $identityId) {  
            return null;  
        }  
    }  
}
```

```

        return \App\Models\IdentityLink::query()
            ->where('active', true)
            ->find($identityId);
    }
}

```

Uso:

```
$identity = currentIdentityLink();
```

## 8. Helper recomendado:

### **hasActiveIdentity()**

### Objetivo

Saber si el usuario autenticado tiene identidad institucional activa.

```

if (! function_exists('hasActiveIdentity')) {
    function hasActiveIdentity(): bool
    {
        return filled(activeIdentityLinkId());
    }
}

```

Uso:

```

if (! hasActiveIdentity()) {
    abort(403, 'No tienes una identidad institucional asociada.');
```

## 9. Uso al crear registros

Cuando un módulo funcional cree registros asociados al usuario actual, debe guardar:

```
'owner_id' => activeIdentityLinkId()
```

Ejemplo:

```
Registro::query()->create([
    'owner_id' => activeIdentityLinkId(),
    'titulo' => $this->titulo,
    'descripcion' => $this->descripcion,
]);
```

## 10. Uso al filtrar registros propios

Para listar registros pertenecientes a la identidad actual:

```
Registro::query()
    ->where('owner_id', activeIdentityLinkId())
    ->get();
```

Ejemplo en Livewire:

```
public function render()
{
    return view('livewire.modulo.index', [
        'registros' => Registro::query()
            ->where('owner_id', activeIdentityLinkId())
            ->latest()
            ->paginate(15),
    ]);
}
```

## 11. Scope recomendado en modelos

Para evitar repetir el filtro en cada consulta, se puede agregar un scope:

```
public function scopeOwnedByCurrentIdentity($query)
{
    return $query->where('owner_id', activeIdentityLinkId());
}
```

Uso:

```
Registro::ownedByCurrentIdentity()->get();
```

## 12. Relación recomendada en modelos funcionales

Si un modelo tiene:

```
owner_id
```

la relación debe apuntar a `IdentityLink`:

```
public function owner()
{
    return $this->belongsTo(IdentityLink::class, 'owner_id', 'id');
}
```

No debe relacionarse directamente con `User`, `Persona` o `Estudiante`, porque la propiedad institucional puede venir de distintas fuentes.

## 13. Ejemplo de modelo funcional

```
use App\Models\IdentityLink;
use Illuminate\Database\Eloquent\Model;

class Registro extends Model
{
    protected $fillable = [
```



```

        'owner_id',
        'titulo',
        'descripcion',
    ];

    public function owner()
    {
        return $this->belongsTo(IdentityLink::class, 'owner_id', 'id');
    }

    public function scopeOwnedByCurrentIdentity($query)
    {
        return $query->where('owner_id', activeIdentityLinkId());
    }
}

```

## 14. Ejemplo de migración para módulo funcional

```

Schema::create('registros', function (Blueprint $table) {
    $table->id();

    /*
     * owner_id referencia lógicamente a identity_links.id.
     * Según la política actual de SIIAA, puede manejarse sin FK física
     * para conservar flexibilidad en importaciones, respaldos y fuentes externas.
     */
    $table->unsignedBigInteger('owner_id')->index();

    $table->string('titulo');
    $table->text('descripcion')->nullable();

    $table->unsignedBigInteger('created_by')->nullable()->index();
    $table->unsignedBigInteger('updated_by')->nullable()->index();

    $table->timestamps();
    $table->softDeletes();
});

```

## 15. Uso en componentes administrativos

En módulos donde un administrador pueda ver todos los registros, se debe distinguir entre:

```
Vista personal
    filtra por owner_id = activeIdentityLinkId()

Vista administrativa
    puede ver todos si tiene permiso
```

Ejemplo:

```
$query = Registro::query();

if (! auth()->user()->can('registros.view_all')) {
    $query->where('owner_id', activeIdentityLinkId());
}
```

## 16. Ventajas de esta arquitectura

### Unifica personas y estudiantes

La misma lógica funciona para:

```
Personal IRyA
Estudiantes SIIAP
Identities futuras
```

### Evita acoplar módulos a tablas específicas

Los módulos no necesitan saber si el propietario viene de:

```
personas
estudiantes
```

Solo necesitan:

```
identity_links.id
```

## Conserva historial institucional

Aunque un usuario cambie, se desactive o deje de estar vigente, los registros pueden conservar su propietario institucional.

## Diferencia acceso de propiedad

Un administrador puede editar un registro sin convertirse en propietario institucional del mismo.

# 17. Regla operativa recomendada

Para todos los módulos futuros:

1. Usar `owner_id` como referencia a `identity_links.id`.
2. Usar `activeIdentityLinkId()` para asignar propietario actual.
3. Usar `currentIdentityLink()` cuando se requiera el modelo completo.
4. Usar `created_by` y `updated_by` para auditoría de usuarios.
5. No usar `users.id` como propietario institucional.
6. No usar `personas.id` ni `estudiantes.id` como `owner` directo, salvo excepción documentada.

# 18. Pendiente técnico a confirmar

Debe verificarse en el middleware real:

```
app/Http/Middleware/ResolveCurrentIdentity.php
```

cómo se está guardando actualmente la identidad activa.

Posibles nombres actuales:

```
session('identity_link_id')  
session('current_identity_id')
```

```
session('current_identity.link_id')
```

La recomendación es normalizar a:

```
session('identity_link_id')
```

y que todo acceso externo pase por:

```
activeIdentityLinkId()  
currentIdentityLink()  
hasActiveIdentity()
```

---

## 19. Resumen breve

La identidad institucional activa del usuario debe obtenerse desde la capa ResolveCurrent Identity. Esa identidad corresponde a un registro de identity\_links. Para módulos funcionales, owner\_id debe apuntar a identity\_links.id. Se recomienda exponer helpers globales como activeIdentityLinkId() y currentIdentityLink() para evitar depender directamente de session('identity\_link\_id') en cada componente. created\_by y updated\_by deben seguir representando al usuario que realizó la acción, mientras owner\_id representa a la identidad institucional propietaria del registro.